

My Chance

Onde seu perfil profissional é o mais importante

Documentação Técnica

Módulo de Algoritmos de Matching

Item 4 — Arquitetura e Principais Tecnologias

Algoritmos documentados neste arquivo:

1. Pontuação Ponderada por Competências
2. Similaridade por Cosseno (Cosine Similarity)

Equipe My Chance — Disciplina de MVP

Davi Lima de Medeiros Israel Buriti Galvão
João Victor Cosme Melo José do Bomfim Truta Neto
Lucas Leonardo Mayer Almeida Lucas Queiroz Correia

Contents

1	Introdução e Contexto	2
2	Algoritmo 1 — Pontuação Ponderada por Competências	3
2.1	Fundamentos Teóricos	3
2.2	Fórmula Matemática	3
2.3	Extensões para o My Chance	3
2.4	Exemplo Numérico Detalhado	4
2.5	Implementação em Python	4
2.6	Vantagens e Limitações	8
3	Algoritmo 2 — Matching por Similaridade Vetorial (Cosine Similarity)	9
3.1	Fundamentos Teóricos	9
3.2	Fórmula Matemática	9
3.3	Construção dos Vetores no My Chance	9
3.3.1	Vocabulário de Competências	10
3.3.2	Exemplo de Vetorização	10
3.4	Versão Ponderada da Cosine Similarity	10
3.5	Implementação em Python	10
3.6	Vantagens e Limitações	13
4	Arquitetura Integrada dos Algoritmos	14
4.1	Visão Geral do Pipeline	14
4.2	Implementação do Pipeline Completo	14

1. Introdução e Contexto

O **My Chance** é uma plataforma de recrutamento às cegas e invertido, cujo objetivo central é reduzir a influência de vieses inconscientes na triagem de candidatos. Em vez de candidatos se candidatarem a vagas, *recrutadores* buscam ativamente perfis no banco de dados, enxergando apenas competências técnicas e experiências — os dados pessoais são revelados somente após o aceite mútuo de uma entrevista.

Para que esse mecanismo funcione com qualidade e justiça, o sistema precisa de um **motor de matching inteligente**: um conjunto de algoritmos que ordene candidatos por compatibilidade com uma vaga de forma precisa, transparente e escalável.

Este documento descreve, de forma didática e detalhada, os dois algoritmos propostos para compor esse motor, incluindo fundamentos matemáticos, implementação em Python e orientações de integração com o restante da arquitetura (Streamlit, SQLite/PostgreSQL, Docker).

Arquitetura em Camadas

A sugestão estratégica para o MVP é combinar os dois algoritmos em sequência:

1. **Pontuação Ponderada** — filtragem rápida e eliminação de candidatos com lacunas críticas.
2. **Cosine Similarity** — ranking principal de compatibilidade.

2. Algoritmo 1 — Pontuação Ponderada por Competências

2.1. Fundamentos Teóricos

A **Pontuação Ponderada por Competências** (*Weighted Skill Scoring*) é um método de ranqueamento baseado em somas ponderadas. É um algoritmo mais simples, mas também o mais transparente e explicável, características essenciais para um MVP que precisará convencer recrutadores.

Intuição

Pense como um recrutador humano preenchendo uma planilha: para cada competência exigida pela vaga, ele atribui uma importância (peso) e avalia o nível do candidato. A nota final é a soma dessas avaliações. Simples, auditável, justo.

2.2. Fórmula Matemática

Dado um candidato C e uma vaga V com n competências, o score é:

Fórmula da Pontuação Ponderada

$$\text{Score}(C, V) = \sum_{i=1}^n w_i \cdot l_i \quad (1)$$

onde:

- $w_i \in \mathbb{R}^+$ — peso da competência i definido pela vaga;
- $l_i \in [0, L_{\max}]$ — nível declarado do candidato na competência i ;
- $L_{\max}$ — nível máximo possível (e.g., 5 em uma escala de 1–5).

Para normalizar o score entre 0 e 1, podemos dividir pelo score máximo teórico:

Score Normalizado

$$\text{Score}_{\text{norm}}(C, V) = \frac{\sum_{i=1}^n w_i \cdot l_i}{\sum_{i=1}^n w_i \cdot L_{\max}} \quad (2)$$

2.3. Extensões para o My Chance

Além da fórmula base, o sistema pode incorporar bônus e penalizações:

Fórmula Estendida

$$\text{Score}_{\text{ext}}(C, V) = \underbrace{\sum_{i=1}^n w_i \cdot l_i}_{\text{base}} + \underbrace{\beta_p \cdot N_p}_{\text{bônus projetos}} + \underbrace{\beta_r \cdot R_r}_{\text{bônus recência}} - \underbrace{\gamma_m \cdot M_o}_{\text{penalização obrig.}} \quad (3)$$

onde:

- N_p — número de projetos reais relacionados; β_p — bônus unitário por projeto;
- R_r — fator de recência da experiência ($\in [0, 1]$, decai com o tempo);
- M_o — número de competências obrigatórias ausentes; γ_m — penalização por ausência.

2.4. Exemplo Numérico Detalhado

Exemplo — Vaga de Analista de Dados

Requisitos da vaga:

Competência	Peso (w_i)	Obrigatória?
Python	5	Sim
SQL	4	Sim
Docker	3	Não
Power BI	2	Não

Perfil do candidato A (nível em escala 0–5): Python = 4, SQL = 5, Docker = 2, Power BI = 0, projetos = 2

Cálculo:

$$\text{Score}_{\text{base}} = (5 \cdot 4) + (4 \cdot 5) + (3 \cdot 2) + (2 \cdot 0) = 46$$

$$\text{Score}_{\text{máx}} = (5 + 4 + 3 + 2) \cdot 5 = 70$$

$$\text{Score}_{\text{norm}} = \frac{46}{70} \approx 0,657$$

$$\text{Bônus projetos} = 2 \cdot 2,0 = +4,0$$

$$\text{Score}_{\text{final}} \approx 0,657 + 0,057 \approx 0,714$$

2.5. Implementação em Python

```

1 from dataclasses import dataclass, field
2 from typing import Optional
3
4
5 @dataclass
6 class Vaga:
7     id: int
8     titulo: str

```

```

9      # Ex: {"python": {"peso": 5, "obrigatoria": True, "nivel_min": 3}}
10      competencias: dict
11
12
13 @dataclass
14 class Candidato:
15     id: int
16     nome_anonimo: str          # ID anonimo no My Chance
17     # Ex: {"python": 4, "sql": 5, "docker": 2}
18     competencias: dict
19     projetos: int = 0          # Projetos reais no portfolio
20     anos_experiencia: float = 0.0
21
22
23 def calcular_fator_recencia(anos: float) -> float:
24     """
25     Decaimento exponencial: experiencias recentes valem mais.
26     Anos 0-1 -> fator ~1.0
27     Anos 5   -> fator ~0.6
28     Anos 10+ -> fator ~0.37
29     """
30     import math
31     return math.exp(-0.1 * anos)
32
33
34 def calcular_score_ponderado(
35     candidato: Candidato,
36     vaga: Vaga,
37     nivel_maximo: int = 5,
38     bonus_por_projeto: float = 2.0,
39     penalizacao_obrigatoria: float = 15.0,
40 ) -> dict:
41     """
42     Calcula o score ponderado de um candidato para uma vaga.
43
44     Retorna um dict com score bruto, normalizado e detalhes.
45     """
46     score_base = 0.0
47     score_maximo = 0.0
48     ausencias_obrigatorias = 0
49     detalhes = {}
50
51     for competencia, config in vaga.competencias.items():
52         peso = config["peso"]
53         obrigatoria = config.get("obrigatoria", False)
54         nivel_candidato = candidato.competencias.get(competencia
55 , 0)
56
57         contribuicao = peso * nivel_candidato
58         score_base += contribuicao

```

```

58     score_maximo += peso * nivel_maximo
59
60     detalhes[competencia] = {
61         "peso": peso,
62         "nivel_candidato": nivel_candidato,
63         "contribuicao": contribuicao,
64     }
65
66     # Verifica ausencia de competencia obligatoria
67     nivel_minimo = config.get("nivel_min", 1)
68     if obligatoria and nivel_candidato < nivel_minimo:
69         ausencias_obrigatorias += 1
70
71     # Score normalizado [0, 1]
72     score_normalizado = score_base / score_maximo if
score_maximo > 0 else 0.0
73
74     # Bonus por projetos
75     bonus_projetos = (candidato.projetos * bonus_por_projeto) /
score_maximo
76
77     # Fator de recencia
78     fator_recencia = calcular_fator_recencia(candidato.
anos_experiencia)
79
80     # Penalizacao por competencias obligatorias ausentes
81     penalizacao = (ausencias_obrigatorias *
penalizacao_obrigatoria) / score_maximo
82
83     # Score final
84     score_final = max(
85         0.0,
86         (score_normalizado + bonus_projetos) * fator_recencia -
penalizacao
87     )
88
89     return {
90         "candidato_id": candidato.id,
91         "score_base_normalizado": round(score_normalizado, 4),
92         "score_final": round(score_final, 4),
93         "ausencias_obrigatorias": ausencias_obrigatorias,
94         "detalhes_por_competencia": detalhes,
95     }
96
97
98 def rankear_candidatos(candidatos: list[Candidato],
99                       vaga: Vaga) -> list[dict]:
100     """Ordena candidatos por score decrescente."""
101     resultados = [
102         calcular_score_ponderado(c, vaga) for c in candidatos
103     ]

```

```

104     return sorted(resultados, key=lambda x: x["score_final"],
105                   reverse=True)
106
107
108 #
109
110 # Exemplo de uso
111 #
112
113 if __name__ == "__main__":
114     vaga_analista = Vaga(
115         id=1,
116         titulo="Analista de Dados",
117         competencias={
118             "python": {"peso": 5, "obrigatoria": True, "
119             nivel_min": 3},
120             "sql": {"peso": 4, "obrigatoria": True, "
121             nivel_min": 2},
122             "docker": {"peso": 3, "obrigatoria": False, "
123             nivel_min": 1},
124             "powerbi": {"peso": 2, "obrigatoria": False, "
125             nivel_min": 1},
126         }
127     )
128
129     candidatos = [
130         Candidato(1, "CAND-001",
131                   {"python": 4, "sql": 5, "docker": 2, "powerbi"
132                   : 0},
133                   projetos=2, anos_experiencia=1.0),
134         Candidato(2, "CAND-002",
135                   {"python": 5, "sql": 4, "docker": 4, "powerbi"
136                   : 3},
137                   projetos=5, anos_experiencia=3.0),
138         Candidato(3, "CAND-003",
139                   {"python": 2, "sql": 1, "docker": 0, "powerbi"
140                   : 5},
141                   projetos=0, anos_experiencia=0.5),
142     ]
143
144     ranking = rankear_candidatos(candidatos, vaga_analista)
145     for pos, r in enumerate(ranking, 1):
146         print(f"{pos}. {r['candidato_id']} "
147               f"| Score: {r['score_final']:.3f} "
148               f"| Ausencias obrig.: {r['ausencias_obrigatorias
149               ']}")

```

Listing 1: Pontuação ponderada com bônus e penalizações

2.6. Vantagens e Limitações

Table 1: Análise da Pontuação Ponderada no projeto

Vantagens	Limitações
Extremamente simples de implementar	Pesos iniciais são subjetivos (recrutador define)
Completamente transparente e auditável	Não captura similaridade estrutural entre perfis
Fácil de explicar em apresentação	Escalas declaradas pelo candidato podem ser inconsistentes
Permite penalização por gaps críticos	Não detecta combinações sinérgicas de competências
Funciona mesmo sem histórico de dados	Ranking linear pode ser simplista para vagas complexas

3. Algoritmo 2 — Matching por Similaridade Vetorial (Cosine Similarity)

3.1. Fundamentos Teóricos

A **Similaridade por Cosseno** (*Cosine Similarity*) é uma medida de similaridade entre dois vetores no espaço euclidiano. É amplamente utilizada em sistemas de recomendação, recuperação de informação e processamento de linguagem natural por uma razão elegante: ela captura a *direção* dos vetores, não apenas suas magnitudes.

Intuição Geométrica

Imagine cada competência como uma dimensão em um espaço vetorial. Tanto o candidato quanto a vaga são representados como pontos nesse espaço. A *Cosine Similarity* mede o **ângulo** entre esses dois pontos — quanto menor o ângulo, mais similares os perfis. Isso significa que um candidato com todas as competências no nível 4 pode ser tão compatível com uma vaga quanto um com todas no nível 2, se a *proporção* for similar.

3.2. Fórmula Matemática

Dados dois vetores **A** (vaga) e **B** (candidato) no $\mathbb{R}^n$:

Similaridade por Cosseno

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \cdot \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \cdot \sqrt{\sum_{i=1}^n B_i^2}} \quad (4)$$

onde θ é o ângulo entre os vetores e o resultado pertence ao intervalo $[-1, 1]$.

Para o caso de competências (valores não negativos), o resultado estará sempre em $[0, 1]$:

Table 2: Interpretação da Cosine Similarity

$\cos(\theta)$	θ	Interpretação
1,00	0	Perfis idênticos
0,85 – 0,99	< 30	Alta compatibilidade
0,60 – 0,84	30 – 53	Compatibilidade moderada
0,30 – 0,59	> 53	Baixa compatibilidade
0,00	90	Sem competências em comum

3.3. Construção dos Vetores no My Chance

O passo mais importante é a **vetorização**: converter perfis textuais em vetores numéricos.

3.3.1. Vocabulário de Competências

Define-se um vocabulário global $\mathcal{V} = \{c_1, c_2, \dots, c_m\}$ com todas as competências cadastradas no sistema. Cada perfil vira um vetor $\mathbf{v} \in \mathbb{R}^m$, onde v_i representa o nível naquela competência (ou 0 se ausente).

3.3.2. Exemplo de Vetorização

Considere $\mathcal{V} = \{\text{Python}, \text{SQL}, \text{Docker}, \text{React}\}$:

Vetorização e Cálculo de Similaridade

Vaga: Python=1, SQL=1, Docker=1, React=0 $\Rightarrow \mathbf{V} = (1, 1, 1, 0)$

Candidato A: Python=1, SQL=1, Docker=0, React=1 $\Rightarrow \mathbf{C}_A = (1, 1, 0, 1)$

Candidato B: Python=5, SQL=4, Docker=4, React=0 $\Rightarrow \mathbf{C}_B = (5, 4, 4, 0)$

Cálculo para Candidato A:

$$\cos(\theta_A) = \frac{(1)(1) + (1)(1) + (1)(0) + (0)(1)}{\sqrt{3} \cdot \sqrt{3}} = \frac{2}{3} \approx 0,667$$

Cálculo para Candidato B:

$$\cos(\theta_B) = \frac{(1)(5) + (1)(4) + (1)(4) + (0)(0)}{\sqrt{3} \cdot \sqrt{57}} = \frac{13}{\sqrt{171}} \approx 0,994$$

Candidato B é mais compatível, mesmo usando escala 0–5 vs 0–1.

3.4. Versão Ponderada da Cosine Similarity

É possível incorporar os pesos da vaga diretamente na similaridade:

Cosine Similarity Ponderada

$$\cos_w(\theta) = \frac{\sum_{i=1}^n w_i \cdot A_i \cdot B_i}{\sqrt{\sum_{i=1}^n w_i \cdot A_i^2} \cdot \sqrt{\sum_{i=1}^n w_i \cdot B_i^2}} \quad (5)$$

Isso combina a expressividade geométrica da Cosine Similarity com os pesos de importância definidos pela vaga.

3.5. Implementação em Python

```
1 import numpy as np
2 import pandas as pd
3 from typing import Optional
4
5
6 def vetorizar_perfil(competencias_perfil: dict,
7                     vocabulario: list[str]) -> np.ndarray:
8     """
9     Converte um dicionario de competencias em vetor numpy.
```

```

10
11     Parametros
12     -----
13     competencias_perfil : {"python": 4, "sql": 3, ...}
14     vocabulario          : lista ordenada de todas as
competencias
15
16     Retorna
17     -----
18     np.ndarray de tamanho len(vocabulario)
19     """
20     return np.array([
21         float(competencias_perfil.get(c, 0)) for c in
vocabulario
22     ])
23
24
25 def cosine_similarity(a: np.ndarray, b: np.ndarray,
26                       pesos: Optional[np.ndarray] = None) ->
float:
27     """
28     Calcula a similaridade por cosseno entre dois vetores.
29     Suporta versao ponderada se 'pesos' for fornecido.
30     """
31     if pesos is not None:
32         # Versao ponderada
33         num = np.sum(pesos * a * b)
34         den = (np.sqrt(np.sum(pesos * a**2)) *
35               np.sqrt(np.sum(pesos * b**2)))
36     else:
37         num = np.dot(a, b)
38         den = np.linalg.norm(a) * np.linalg.norm(b)
39
40     if den == 0:
41         return 0.0
42     return float(num / den)
43
44
45 def construir_matriz_scores(candidatos: list[dict],
46                             vaga: dict,
47                             vocabulario: list[str]) -> pd.
DataFrame:
48     """
49     Monta o ranking completo de candidatos para uma vaga.
50
51     Parametros
52     -----
53     candidatos : lista de dicts com 'id', 'competencias'
54     vaga        : dict com 'id', 'competencias', 'pesos'
55     vocabulario : lista de todas as competencias do sistema
56

```

```

57     Retorna
58     -----
59     DataFrame ordenado por similaridade decrescente
60     """
61     vetor_vaga = vetorizar_perfil(
62         vaga["competencias"], vocabulario
63     )
64     vetor_pesos = vetorizar_perfil(
65         vaga.get("pesos", {}), vocabulario
66     )
67     # Se nao houver pesos, usa vetor uniforme
68     if vetor_pesos.sum() == 0:
69         vetor_pesos = None
70
71     resultados = []
72     for c in candidatos:
73         vetor_candidato = vetorizar_perfil(
74             c["competencias"], vocabulario
75         )
76         sim = cosine_similarity(
77             vetor_vaga, vetor_candidato, vetor_pesos
78         )
79         resultados.append({
80             "candidato_id": c["id"],
81             "similaridade": round(sim, 4),
82             "angulo_graus": round(
83                 np.degrees(np.arccos(np.clip(sim, -1, 1))), 1
84             ),
85         })
86
87     df = pd.DataFrame(resultados)
88     return df.sort_values("similaridade",
89                           ascending=False).reset_index(drop=True)
90
91
92     # -----
93
94     # Exemplo de uso integrado
95     # -----
96
97     if __name__ == "__main__":
98         vocabulario = ["python", "sql", "docker", "react",
99                       "git", "powerbi", "java"]
100
101         vaga = {
102             "id": 1,
103             "competencias": {"python": 1, "sql": 1, "docker": 1,
104                             "git": 1},

```

```

103     "pesos":          {"python": 5, "sql": 4, "docker": 3,
104                       "git": 2},
105 }
106
107 candidatos = [
108     {"id": "CAND-001",
109      "competencias": {"python": 4, "sql": 3, "docker": 2,
110                       "git": 5, "react": 1}},
111     {"id": "CAND-002",
112      "competencias": {"python": 5, "sql": 5, "docker": 4,
113                       "git": 4}},
114     {"id": "CAND-003",
115      "competencias": {"react": 5, "java": 4, "powerbi": 3}},
116     {"id": "CAND-004",
117      "competencias": {"python": 2, "sql": 2, "docker": 1,
118                       "git": 3, "powerbi": 2}},
119 ]
120
121 ranking = construir_matriz_scores(candidatos, vaga,
122                                  vocabulario)
123 print(ranking.to_string(index=False))

```

Listing 2: Cosine Similarity para matching de vagas e candidatos

3.6. Vantagens e Limitações

Table 3: Análise da Cosine Similarity no projeto

Vantagens	Limitações
Invariante à escala dos níveis	Não penaliza explicitamente competências obrigatórias ausentes
Captura estrutura dos perfis com precisão	Sensível à esparsidade (perfis com poucas competências)
Escala facilmente para milhares de candidatos	Exige vocabulário de competências bem definido
Compatible com embeddings de texto no futuro	Não considera experiência temporal ou contexto
Amplamente validado em sistemas de recomendação	Candidatos com listas curtas podem ser subavaliados

4. Arquitetura Integrada dos Algoritmos

4.1. Visão Geral do Pipeline

Os dois algoritmos são complementares e devem ser usados em sequência, formando um **pipeline de matching em duas etapas**:

Etapa	Algoritmo	Função	Output
1	Pontuação Ponderada	Filtragem rápida	Lista reduzida
2	Cosine Similarity	Ranking principal	Score de 0 a 1

4.2. Implementação do Pipeline Completo

```

1 class MotorDeMatching:
2     """
3     Pipeline integrado: Pontuacao Ponderada -> Cosine Similarity
4     """
5
6     def __init__(self, vocabulario: list[str],
7                  limiar_ponderado: float = 0.4):
8         self.vocabulario = vocabulario
9         self.limiar_ponderado = limiar_ponderado # Corte na
10                                                    etapa 1
11
12     def rankear(self,
13                candidatos: list[dict],
14                vaga: dict,
15                top_n: int = 10) -> list[dict]:
16         """
17         Executa o pipeline completo e retorna os top-N
18         candidatos.
19         """
20         # ETAPA 1: Filtragem por Pontuacao Ponderada
21         candidatos_filtrados = []
22         for c in candidatos:
23             r = calcular_score_ponderado(
24                 Candidato(c["id"], c["id"],
25                          c["competencias"],
26                          c.get("projetos", 0),
27                          c.get("experiencia", 0.0)),
28                 Vaga(vaga["id"], "", vaga["competencias_config"]
29             )
30             # Descarta candidatos com ausencias obrigatorias
31             if r["ausencias_obrigatorias"] == 0:
32                 candidatos_filtrados.append(c)

```

```
32     if not candidatos_filtrados:
33         return []
34
35     # ETAPA 2: Ranking por Cosine Similarity
36     ranking = construir_matriz_scores(
37         candidatos_filtrados, vaga, self.vocabulario
38     )
39
40     return ranking.head(top_n).to_dict(orient="records")
```

Listing 3: Pipeline completo de matching

Referências

1. SALTON, G.; MCGILL, M. J. *Introduction to Modern Information Retrieval*. New York: McGraw-Hill, 1983.
2. SCIKIT-LEARN DEVELOPERS. *sklearn.metrics.pairwise.cosine_similarity*. Disponível em: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.pairwise.cosine_similarity.html
3. PEDREGOSA, F. et al. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, v. 12, p. 2825–2830, 2011.